

K plus proches voisins (KNN)

Laurent Amanton



- ① Qu'est-ce que KNN ?
- ② Principe du KNN
- ③ Classification du voisin le plus proche
- ④ L'algorithme du KNN
- ⑤ La distance : essentiel
- ⑥ Premier Exemple : 2-KNN
- ⑦ Deuxième Exemple : 3-KNN
- ⑧ Influence du facteur K
- ⑨ Normalisation des données
- ⑩ L'implémentation du KNN en python
- ⑪ Points forts / Points faibles

Les algorithmes d'apprentissage supervisé

① Les régressions

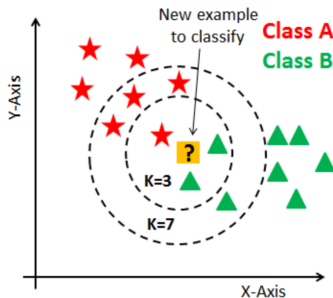
- Régression linéaire simple
- Régression linéaire multiple
- Régression polynomiale

② Les classifications

- Régression logistique
- **K plus proches voisins (KNN)**
- Machine à vecteurs de support (SVM)
- Naïve Bayes classifieur
- Arbres de décisions

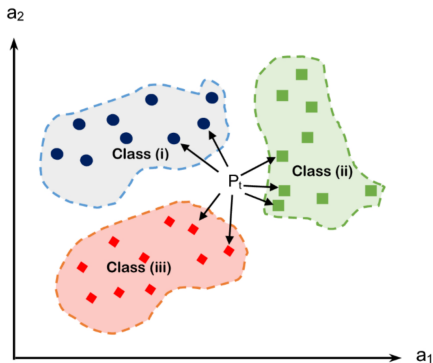
Qu'est-ce que KNN ?

- **KNN** : algorithme de ML supervisé qui peut être utilisé pour les **problèmes prédictifs de classification** et pour la régression.
- Il s'agit d'un algorithme d'apprentissage paresseux (**Lazy Learning Algorithm**) : il n'apprend pas immédiatement de l'ensemble de formation, mais il stocke l'ensemble de données et au moment de la classification puis effectue une action sur l'ensemble de données.



Qui se ressemble s'assemble.

- En d'autres termes, les choses qui se ressemblent sont susceptibles d'avoir des **propriétés** identiques.
- Nous pouvons utiliser ce principe pour classer les données en les plaçant dans la catégorie ayant **les voisins les plus similaires**, ou "les plus proches".

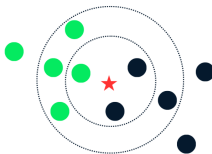


- Les classificateurs des voisins les plus proches sont définis par leur caractéristique de classer les éléments non étiquetés en leur attribuant **la classe des exemples étiquetés les plus similaires**.
- Malgré la simplicité de cette idée, les méthodes du plus proche voisin sont extrêmement puissantes. Ils ont été utilisés avec succès pour :
 - la vision par ordinateur, y compris la reconnaissance optique de caractères et la reconnaissance faciale dans les images fixes et vidéo.
 - la prédiction des recommandations de films Netflix.
 - l'identification des modèles dans les données génétiques pour détecter des protéines ou des maladies.

L'algorithme du KNN

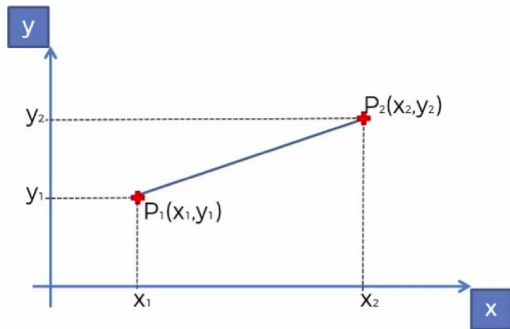
L'algorithme **KNN** commence par un ensemble de données d'entraînement composé d'exemples classés en plusieurs catégories, étiquetées par une variable nominale.

- On dispose d'un ensemble de données de test contenant des exemples non étiquetés qui présentent les mêmes fonctionnalités que les données d'entraînement.
- Pour chacun d'eux **kNN** identifie **k enregistrements** dans les données d'apprentissage qui sont les plus “proches” en termes de similarité. k est un nombre entier défini à l'avance.
- L'instance de test non étiquetée se voit attribuer la classe de la **majorité** des k voisins les plus proches.



Calcul de la distance

- La localisation des voisins les plus proches nécessite une fonction de distance ou une formule qui mesure la similarité entre deux instances.
 - Il existe de nombreuses façons différentes de calculer la distance.
 - Traditionnellement, l'algorithme kNN utilise **la distance euclidienne**, qui est la distance que l'on mesurerait si l'on pouvait utiliser une règle pour relier deux points.



$$\text{Euclidean Distance between } P_1 \text{ and } P_2 = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

- La distance euclidienne entre deux exemples X et Y :

$$X = [x_1, x_2, x_3, \dots, x_n]$$

$$Y = [y_1, y_2, y_3, \dots, y_n]$$

- est défini comme :

$$D(X, Y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

Distances

Euclidienne

$$\sqrt{\sum_{i=1}^k (x_i - y_i)^2}$$

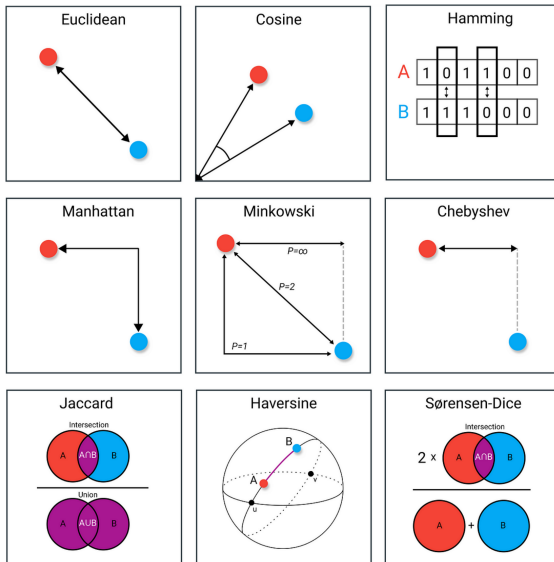
Manhattan

$$\sum_{i=1}^k |x_i - y_i|$$

Minkowski

$$\left(\sum_{i=1}^k (|x_i - y_i|)^q \right)^{1/q}$$

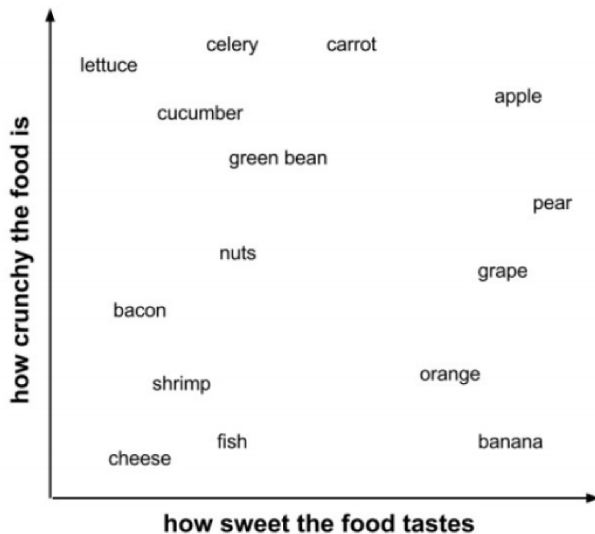
Fonctions de distance



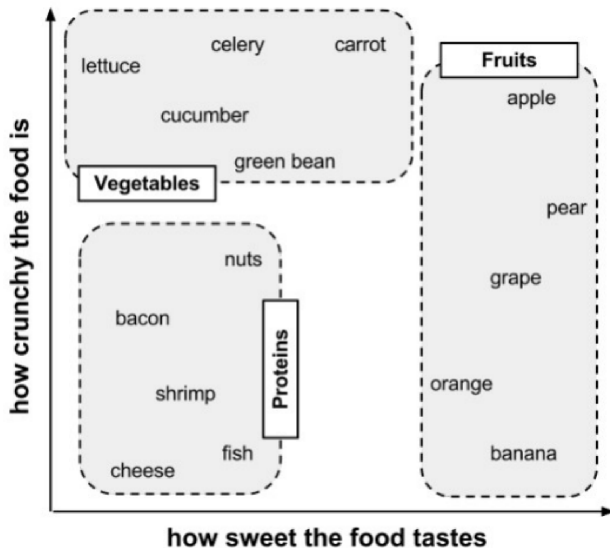
Premier Exemple : 2-KNN

ingredients	goût sucré	croquant	type
apple	10	9	fruit
bacon	1	4	protéine
banana	10	1	fruit
carrot	7	10	légume
celery	3	10	légume
cheese	1	1	protéine

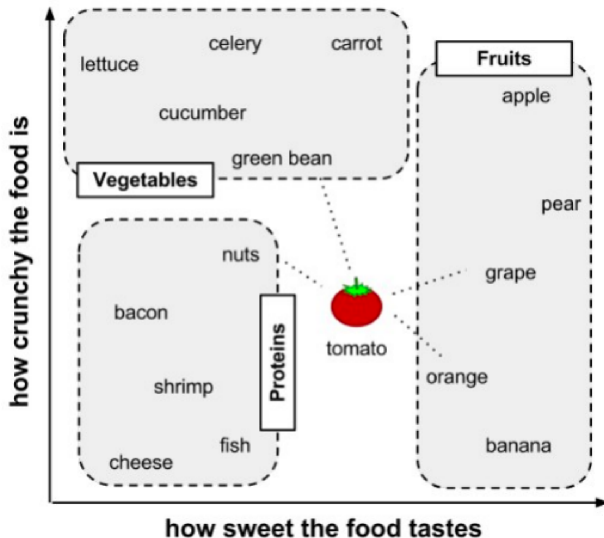
Premier Exemple : 2-KNN



Premier Exemple : 2-KNN



Premier Exemple : 2-KNN



Premier Exemple : 2-KNN

ingredients	goût sucré	croquant	type	distance (tomate)
raisin	8	5	fruit	$\text{sqrt}((6-8)^2 + (4-5)^2) = 2.2$
haricot vert	3	7	légume	$\text{sqrt}((6-3)^2 + (4-7)^2) = 4.2$
noix	3	6	protéine	$\text{sqrt}((6-3)^2 + (4-6)^2) = 3.6$
orange	7	3	fruit	$\text{sqrt}((6-7)^2 + (4-3)^2) = 1.4$

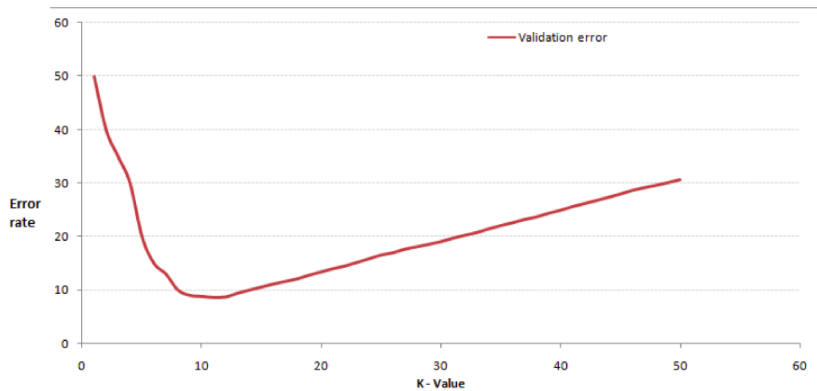
Customer	Age	Income	No. credit cards	Class
Jules	35	35K	3	No
Pierre	22	50K	2	Yes
Lucie	63	200K	1	No
Lisa	59	170K	1	No
Noa	25	40K	4	Yes
Mélina	37	50K	2	YES

Distance from John

$\sqrt{(35-37)^2 + (35-50)^2 + (3-2)^2} = 15.16$
$\sqrt{(22-37)^2 + (50-50)^2 + (2-2)^2} = 15$
$\sqrt{(63-37)^2 + (200-50)^2 + (1-2)^2} = 152.23$
$\sqrt{(59-37)^2 + (170-50)^2 + (1-2)^2} = 122$
$\sqrt{(25-37)^2 + (40-50)^2 + (4-2)^2} = 15.74$

- Décider quel nombre de voisins à utiliser pour kNN détermine dans quelle mesure le modèle se généralisera aux données futures.
 - L'équilibre entre le **sur-ajustement** et le **sous-ajustement** des données d'entraînement est le problème de *compromis biais-variance*.
 - Le choix d'un k élevé réduit l'impact ou la variance provoquée par les données bruitées, mais peut biaiser l'apprenant au point qu'il court le risque d'ignorer des modèles petits mais importants.

Influence du facteur K



- En pratique, le choix de k dépend de la difficulté du concept à apprendre et du nombre d'enregistrements dans les données d'entraînement.
 - Généralement, k est compris entre 3 et 10. Une pratique courante consiste à définir **k égal à la racine carrée du nombre d'exemples d'apprentissage**.
 - Dans un classificateur où il y a 15 exemples dans les données d'entraînement, nous pourrions définir $k = 4$, car la racine carrée de 15 est 3,87.

- Différentes caractéristiques peuvent avoir différentes échelles de mesure. Par exemple, **poids du patient** en kg (plage [50 200]) par rapport aux **valeurs de protéines sanguines** en ng/dL (plage [-3,3]).
- **Conséquences** : Le poids du patient aura une bien plus grande influence sur la distance entre les échantillons, ce qui peut affecter les performances du classificateur.
- Si les plages de caractéristiques diffèrent, les caractéristiques avec des valeurs plus élevées domineront la décision.

En général, les valeurs des caractéristiques sont normalisées avant le calcul de la distance.

- La méthode traditionnelle de redimensionnement des caractéristiques pour kNN est la normalisation minmax.
 - Ce processus transforme une caractéristique de telle sorte que toutes ses valeurs se situent dans une plage comprise entre 0 et 1. La formule pour normaliser une caractéristique est la suivante. Essentiellement, la formule soustrait le minimum de la caractéristique X de chaque valeur et divise par la plage de X :

$$X_{\text{new}} = \frac{X - \min(X)}{\max(X) - \min(X)}$$

- Les valeurs de caractéristiques normalisées peuvent être interprétées comme indiquant dans quelle mesure, de 0 pour cent à 100 pour cent, la valeur d'origine s'est située dans la plage comprise entre le minimum et le maximum d'origine.

Dataset

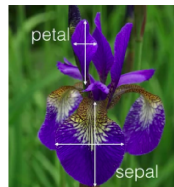
Un petit projet pour débiter consiste à **classifier des fleurs d'iris**.

- Les attributs sont **numériques**, donc aucune difficulté pour gérer les données.
- Il s'agit d'un problème de classification permettant d'utiliser un algorithme d'apprentissage supervisé plus simple.
- Il s'agit d'un problème de **classification multi-classes** (multi-nominal) qui peut nécessiter un traitement spécialisé.
- Il n'a que 4 attributs et 150 lignes, ce qui signifie qu'il est petit et peu gourmand en mémoire.
- Tous les attributs numériques sont dans les **mêmes unités** et **la même échelle**, aucune mise à l'échelle n'est nécessaire.

Dataset IRIS



```
5.1,3.5,1.4,0.2,Iris-setosa
4.9,3.0,1.4,0.2,Iris-setosa
4.7,3.2,1.3,0.2,Iris-setosa
4.6,3.1,1.5,0.2,Iris-setosa
5.0,3.6,1.4,0.2,Iris-setosa
5.4,3.9,1.7,0.4,Iris-setosa
4.6,3.4,1.4,0.3,Iris-setosa
5.0,3.4,1.5,0.2,Iris-setosa
4.4,2.9,1.4,0.2,Iris-setosa
4.9,3.1,1.5,0.1,Iris-setosa
5.4,3.7,1.5,0.2,Iris-setosa
4.8,3.4,1.6,0.2,Iris-setosa
4.8,3.0,1.4,0.1,Iris-setosa
```



Iris Versicolor

Iris Setosa

Iris Virginica

Lecture du Dataset

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

# URL to read the dataset
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data"

# Assign column names to the dataset
names = ['sepal-length', 'sepal-width',
         'petal-length', 'petal-width', 'Class']

# Read dataset to pandas dataframe
dataset = pd.read_csv(url, names=names)

print(dataset.head())
```

Prétraitement

```
# Input variables
X = dataset.iloc[:, :-1].values
# Output variable
y = dataset.iloc[:, 4].values

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20)

# Feature scaling
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaler.fit(X_train)
X_train = scaler.transform(X_train)
X_test = scaler.transform(X_test)
```

K Neighbors Classifier

```
from sklearn.neighbors import KNeighborsClassifier
classifier = KNeighborsClassifier(n_neighbors=5)
classifier.fit(X_train, y_train)

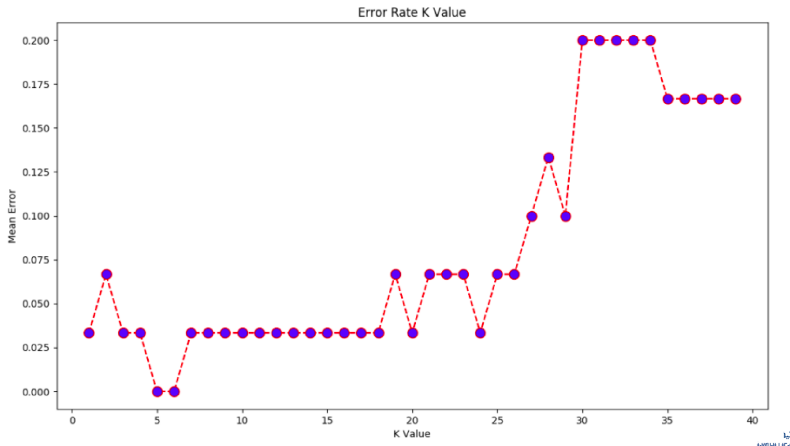
y_pred = classifier.predict(X_test)

from sklearn.metrics import classification_report, \
    confusion_matrix, accuracy_score
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
print(accuracy_score(y_test, y_pred) * 100)
```

Recherche d'erreur avec k modifié

```
error = []  
# Calculating error for K values between 1 and 40  
for i in range(1, 40):  
    knn = KNeighborsClassifier(n_neighbors=i)  
    knn.fit(X_train, y_train)  
    pred_i = knn.predict(X_test)  
    error.append(np.mean(pred_i != y_test))  
  
plt.figure(figsize=(12, 6))  
plt.plot(range(1, 40), error, color='red', linestyle='dashed',  
         marker='o', markerfacecolor='blue', markersize=10)  
plt.title('Error Rate K Value')  
plt.xlabel('K Value')  
plt.ylabel('Mean Error')  
plt.show()
```

Taux d'erreur



En conclusion :

Points forts de KNN

- Très simple et intuitif.
- Peut être appliqué aux données d'une distribution quelconque.
- Bonne classification si le nombre d'échantillons est suffisamment grand.

Points faibles de KNN

- Prend plus de temps pour classer un nouvel exemple : il faut calculer et comparer la distance entre le nouvel exemple et tous les autres exemples.
- Choisir une **bonne valeur de k** peut être délicat.
- la précision dépend du nombre d'échantillons étiquetés.